

1. Introduction

I am presenting my course work in object-oriented programming on the topic “The Game Azure Fable”. My work is about creating a desktop 2D maze game using C# and WPF. In the game, the hero moves through the field, looks for a key, opens a portal, avoids enemies, and can restore health with hearts.

The main game logic is based on OOP principles: each object has its own data and behavior.

2. Goals and Tasks

The goal of the course work is to create the application “Azure Fable”, a 2D maze game with a graphical interface.

To reach this goal, I analyzed the logic of maze games using Pac-Man as an example, designed the structure of the application, described the main processes with flowcharts and UML diagrams, created the hero, enemies, items, maze, and their interaction, and tested the main game scenarios.

3. Overview of Algorithms and Methods

The game is based on the idea of classic maze games: there is a limited field, a controlled character, items, and dangerous objects.

The game field is made as a grid of cells: some cells are walls, and some cells are floor cells. Before each movement, the program checks if the hero can move to the chosen cell.

The hero moves after key presses, and enemies move by a timer. Collisions are checked by coordinates.

The game has two types of enemies: `StandingEnemy`, which stays in one place, and `Ghost`, which moves in a random available direction.

The game state is described by the values `Playing`, `Paused`, `Win`, `GameOver`.

The diagram shows that the program consists of the graphical interface module, the game logic module, and the block of game resources and models. The interface shows the game and controls the screens. The game logic controls the field structure, hero movement, enemy movement, and helper services. The resources and models contain game objects and images.

4. Application Structure

The application architecture is based on the MVVM principle.

The project is divided into main parts: `Models`, `Services`, `Assets`, `Views`, `ViewModels`, and `Converters`.

Models contain the main game objects: the hero, enemies, items, cells, maze, and helper models.

Services are responsible for the game logic: game control, maze generation, enemy movement, collision checking, and item spawning.

Views contain the application screens, and ViewModels connect the interface with the game logic.

Converters contains helper converters for showing data in the interface.

5. OOP Principles in the Game "Azure Fable"

This is the main part of the work. The game uses four main OOP principles: abstraction, encapsulation, inheritance, and polymorphism. They are used not only in theory, but also directly in the class structure and in the game logic.

5.1 Inheritance

Inheritance allows common code to be placed in parent classes, while child classes contain only their differences.

The game has three main inheritance branches: cells, moving objects, and items.

For example, Wall and Floor inherit from Cell, but they implement the possibility of entering the cell in different ways.

Hero, Ghost, and StandingEnemy have a common base through the classes Unit and Enemy.

The items Key, Heart, and Portal inherit from Item, but each item has its own interaction with the hero.

So, inheritance removes code duplication and makes the class structure logical.

Inheritance allows common code to be placed in parent classes, while child classes contain only their differences. The game has three main inheritance branches.

The first branch is cells: GameObject -> Cell -> Wall and Floor. In the CanEnter method, Wall returns false, so entry is not allowed. Floor returns true, so entry is allowed.

The second branch is moving objects: GameObject -> Unit -> Hero and Enemy. Hero gets health and basic methods, and adds HasKey, Move, and CollectKey. Enemy gives a common base for enemies through the Move and Interact methods. Ghost and StandingEnemy inherit from Enemy.

The third branch is items: GameObject -> Item -> Key, Heart, and Portal. All items have common coordinates, activity, name, and image, but each item implements Interact in its own way: Key gives the key, Heart heals, and Portal checks the key and can finish the game with a win.

So, inheritance removes code duplication and makes the class structure logical.

5.2 Polymorphism

Polymorphism allows the same methods to be called for different objects, but the result depends on the specific class.

For example, the method Interact() is called in the same way for the key, heart, and portal, but it does different actions: the key gives the hero a key, the heart restores health, and the portal checks if the hero has the key and can finish the game with a win.

In the same way, the method Move() is called for enemies through the base type Enemy, but Ghost moves, and StandingEnemy stays in place.

Thanks to polymorphism, GameEngine and services do not need many conditions for each type of object.

Polymorphism in the project helps to avoid many type checks.

The clearest example is the Interact() method. For the items Key, Heart, and Portal, it is called in the same way through the IInteractable interface, but it does different actions: collecting the key, healing, or checking for victory. In the same way, for the enemies Ghost and StandingEnemy, CollisionService calls enemy.Interact(hero, maze, random). But StandingEnemy deals damage and becomes inactive, while Ghost deals damage and moves to another cell. In both cases, one common method is used without checking the exact object type.

Polymorphism is also shown in the Move() method for enemies. EnemyLogic works with a list of Enemy objects and calls enemy.Move(maze, random). But the exact movement is decided by the enemy class itself: Ghost moves through the maze, and StandingEnemy stays in place.

The second example is the CanEnter() method in Cell classes. In Wall it returns false, and in Floor it returns true. The Maze class simply calls GetCell(x, y).CanEnter() without checking the cell type.

The third example is programming through service-level interfaces: GameEngine receives IEnemyLogic, ICollisionService, and IItemSpawnService through the constructor and works with them as interfaces.

Thanks to polymorphism, GameEngine and CollisionService do not need many conditions for each type of item, cell, or enemy. This supports the Open/Closed Principle: a new type can be added as a separate class with its own method implementation, without changing the main engine logic.

5.3 Abstraction

Abstraction helps to find common features of different objects.

The base is the abstract class GameObject, which contains coordinates, activity, name, image path, and basic methods for changing position and deactivation.

Other abstract classes are created from it: Unit, Item, and Cell.

Thanks to abstraction, common properties do not need to be repeated in every class, and the interface and logic can work with different objects through one common type.

Abstraction helps to find common features of objects. In the game, this base is the abstract class GameObject. It does not describe a specific hero, enemy, or item. It contains common properties: coordinates X and Y, activity IsActive, name Name, image path ImagePath, and the methods SetPosition, MoveBy, and Deactivate.

Other abstract classes are made from GameObject. Unit describes "living" objects that have Health, MaxHealth, IsAlive, Heal, and TakeDamage. Item describes items that the hero can interact with, so it has the Interact method. Cell describes maze cells and the CanEnter method, which checks if a cell can be entered.

Thanks to abstraction, coordinates, activity, images, and basic logic do not need to be repeated in every class. Also, GameViewModel can work with different game objects through the common GameObject type.

5.4 Encapsulation

Encapsulation protects the inner state of an object from wrong changes.

In the GameObject class, coordinates have private setters, so the position changes only through the methods SetPosition or MoveBy.

In the Unit class, health is also not changed directly, but only through the methods Heal and TakeDamage.

This allows the program to control position and health changes and prevents wrong coordinates or incorrect health values.

Encapsulation protects the inner state of an object from wrong changes. In the `GameObject` class, the coordinates `X` and `Y` have private set parts, so the position changes only through `SetPosition` or `MoveBy`. `SetPosition` checks that the coordinates are not negative and tells the interface that properties have changed.

In the `Unit` class, `Health` is also not changed directly. For this, the methods `Heal` and `TakeDamage` are used. `TakeDamage` reduces health, and when health reaches zero, it deactivates the object. `Heal` restores health, but does not allow it to be higher than `MaxHealth`.

So, every change of position or health goes through controlled methods. This does not allow the hero to be placed in a wrong position by accident, to bypass the wall check, or to set a wrong health value.

6. Interfaces

Interfaces in the project separate the contract from the exact implementation.

`IInteractable` describes objects that the hero can interact with.

`IMaze` describes the main abilities of the maze: getting a cell, adding items, and checking if a cell can be entered.

`IGameEngine` describes game control: start, stop, pause, resume, and hero movement.

There are also interfaces for services: collision checking, enemy movement, and item spawning.

Thanks to interfaces, the code is easier to extend and change.

Interfaces have a separate role in the project, because they separate the contract from the exact implementation.

`IInteractable` describes an object that the hero can interact with. It contains the `IsActive` property and the `Interact(hero)` method. Items implement it through the `Item` class.

`IMaze` describes the main abilities of the maze: getting a cell, adding items, and checking entry into a cell. It is implemented by the `Maze` class, and services work with the maze through `IMaze`.

`IGameEngine` describes game control: start, stop, pause, resume, and hero movement. `GameViewModel` works with the engine through this interface and does not depend on the exact `GameEngine` class.

The service level also has `ICollisionService`, `IEnemyLogic`, and `IItemSpawnService`. They separate the game logic: collisions, enemy movement, and item spawning are processed separately.

Thanks to interfaces, the code is easier to extend and change, because classes depend not on exact implementations, but on clear contracts.

7. Architectural Decisions

The project uses several architectural decisions that made the code more structured.

All `ViewModel` classes inherit from `ViewModelBase`, which implements `INotifyPropertyChanged`, so the interface updates automatically when data changes.

`RelayCommand` is used to process buttons. It connects actions in the interface with methods in the `ViewModel`.

The creation of the game engine is moved to `GameEngineFactory`, so the engine creation logic is not mixed with the interface.

`DispatcherTimer` is used for enemy movement, and its interval depends on the game difficulty.

The project also shows the SOLID principles: each service has its own responsibility, new enemy or item types can be added without changing the main logic, and `GameEngine` depends on abstractions.

Finally, the project shows the SOLID principles: each service has one responsibility (Single Responsibility Principle), a new enemy or item type is added without changing `GameEngine` (Open/Closed Principle), any child class of `Enemy` can work as the base type `Enemy` (LSP ? Liskov Substitution Principle), service interfaces are small and focused (Interface Segregation Principle), and `GameEngine` depends on abstractions, not on exact classes (Dependency Inversion Principle).

8. Interface Demonstration

After launching the application, the player gets to the main menu. It has the main buttons: "Start Game", "Settings", "Help", and "Exit". From the main menu, the user can go to the game, change settings, or view short information about controls.

In the settings window, the player can choose the difficulty: easy, normal, or hard. Difficulty affects the number of enemies and the speed of their movement, so the gameplay becomes easier or harder.

The main game screen contains a maze with the hero, enemies, and items. The hero moves through the cells of the field, collects the key, can pick up a heart to restore health, and after that can go to the portal. The hero's state is shown as hearts, and the key is shown by a separate indicator.

The application also has helper screens: help with controls, pause during the game, exit confirmation, and separate win and loss screens. This makes the interface more complete and clear for the user.

9. Conclusions

As a result of the course work, the application "Azure Fable" was developed. It is a 2D maze game with a graphical interface in C# and WPF. The game has a grid maze, a game loop, two types of enemies, three types of items, four game states, and the main screens.

The main point for the topic of the work is that OOP was used as the base of the program structure. Abstraction selected the common types GameObject, Unit, Item, and Cell. Encapsulation protected the coordinates, health, and activity of objects. Inheritance removed code duplication. Polymorphism allowed GameEngine and services to work with different objects through common methods and interfaces.

10. Completion

The report is finished. Thank you for your attention!

All paths are relative to the AzureFable project. Code is most important in sections 3, 5, 6, and 7. Interface screens are best shown in sections 1 and 8.

1. Introduction

- Open:** Run the application, or open Views/MenuView.xaml.
- Show:** Show the game title and the main menu buttons.
- Say:** This is the desktop 2D maze game Azure Fable. It is created in C# and WPF.

2. Goals and Tasks

- Open:** Show Solution Explorer.
- Show:** Show the main folders: Models, Services, Views, ViewModels, Assets.
- Say:** The project is divided into separate parts, so every part has its own responsibility.

3. Overview of Algorithms and Methods

- Open:** Open Models/Maze.cs and Services/GameEngine.cs.
- Show:** Show CanEnter, MoveHero, CheckItemInteraction, and CheckEnemyInteraction.
- Say:** Maze checks if a cell can be entered. GameEngine controls hero movement, item interaction, and enemy collision.

4. Application Structure

- Open:** Open ViewModels/GameViewModel.cs and Views/GameView.xaml.
- Show:** Show that GameViewModel gives data to the screen, and GameView.xaml displays cells, objects, hearts, and key status.
- Say:** This shows the MVVM structure: the View displays the interface, and the ViewModel connects the interface with game logic.

5.1 Inheritance

- Open:** Open Models/GameObject.cs, Unit.cs, Hero.cs, Enemy.cs, and Item.cs.
- Show:** Show the chains GameObject -> Unit -> Hero / Enemy and GameObject -> Item -> Key / Heart / Portal.
- Say:** Inheritance helps avoid code duplication, because common fields and methods are placed in parent classes.

5.2 Polymorphism

- Open:** Open Models/Key.cs, Heart.cs, Portal.cs, Ghost.cs, and StandingEnemy.cs.
- Show:** Show the same methods Interact() and Move(), but with different implementations.
- Say:** The same method is called, but different classes have different behavior.

5.3 Abstraction

- Open:** Open Models/GameObject.cs, Cell.cs, Unit.cs, and Item.cs.
- Show:** Show abstract classes and common properties: X, Y, IsActive, Name, ImagePath.
- Say:** These abstract classes describe common features, not one exact object.

5.4 Encapsulation

- Open:** Open Models/GameObject.cs and Models/Unit.cs.
- Show:** Show private setters for X, Y, and Health, and the methods SetPosition, MoveBy, Heal, and TakeDamage.
- Say:** The object state cannot be changed directly. It changes only through controlled methods.

6. Interfaces

- Open:** Open Models/IInteractable.cs, Models/IMaze.cs, and Services/IGameEngine.cs.
- Show:** Show method signatures, for example Interact, CanEnter, Start, Pause, Resume, and MoveHero.
- Say:** Interfaces describe what a class must do, but not how it does it. This makes the code easier to extend.

7. Architectural Decisions

- Open:** Open ViewModels/ViewModelBase.cs, RelayCommand.cs, Services/GameEngineFactory.cs, and Services/GameEngine.cs.
- Show:** Show INotifyPropertyChanged, ICommand, GameEngineFactory, and DispatcherTimer.
- Say:** These classes support MVVM: ViewModel updates the interface, commands process buttons, and the timer moves enemies.

8. Interface Demonstration

- Open:** Run the application.
- Show:** Show main menu, settings, game screen, help, pause, win screen, or game over screen.
- Say:** Now I will show how these classes work in the actual interface.

9. Conclusions

- Open:** Do not open new code. Keep the game screen or return to GameObject.cs.
- Show:** Briefly remind the committee that the project uses abstraction, encapsulation, inheritance, and polymorphism.
- Say:** The main result is a working 2D maze game where OOP is used as the base of the program structure.

10. Completion

- Open:** Do not open code.
- Show:** Finish calmly.
- Say:** This is the end of my report. Thank you for your attention.